

Fake it in software - as little hardware as possible

Fake it in software

RISC OS 5 boots on an emulated Raspberry Pi 4 by modelling almost none of the Pi's hardware. It answers the operating system's questions instead.

The fork runs the stock RISC OS Open 5.30 Pi ROM on QEMU's `raspi4b` machine, with the Cortex-A72 in AArch32 mode. What makes that tractable is one design rule, recorded in the project's design notes in the owner's own words:

rather than emulating videocore; we should emulate the functions needed by risc os ... let's be as soft and fake as possible and emulate hardware only if we are desperate ... we probably do need interrupts though, the whole system is running off timers.

That sentence is the subject of this document. It explains what gets modelled faithfully — very little — what gets answered instead, how a RISC OS module talks to the host without a single forged CPU register, and what the approach genuinely cost.

These documents

Chapter	What it covers
1. Don't build the machine you can borrow	Why QEMU and a Pi 4, and the tracing method that found every blocker
2. As soft and fake as possible	The rule, its origin, and the first five fakes
3. What the OS genuinely waits for	Interrupts and time from one host thread, and what stayed emulated
4. We are the GPU	Sound and the pointer, by impersonating the firmware's own protocols

The chapters, continued

Chapter	What it covers
5. A blitter RISC OS never had	A device with no silicon behind it, and a module that knows when to decline
6. A doorbell and a filing system	The guest–host channel, and why the host walks the guest's MMU
7. Booting from the host	Splicing a module into a stock ROM, and an honest reading of what it bought
8. The bill	Silent failures, module coupling, lost fidelity, and the method that kept it honest

At a glance

Machine	QEMU raspib4b, Cortex-A72 in AArch32, under TCG
Guest	the stock RISC OS Open 5.30 Pi ROM — never rebuilt, never patched in place
Emulated faithfully	the CPU, interrupt and FIQ plumbing, and the devices the ROM drives by register
Answered instead	firmware power, VCHIQ, sound, the pointer, EDID, CMOS, timers, drawing, files
Guest–host channels	firmware protocols the ROM already speaks, and two paravirtual doorbells

The shortest possible summary

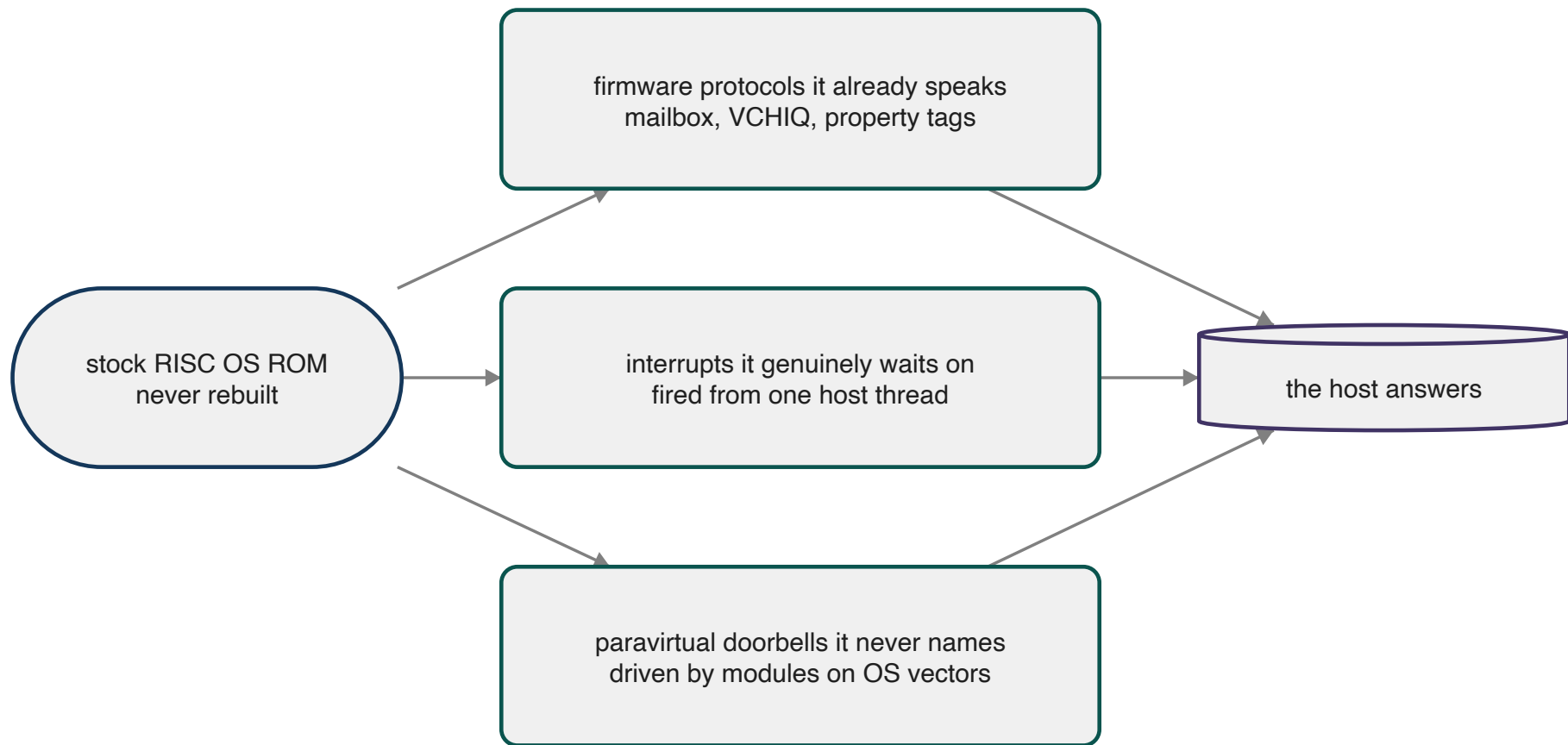
A real Raspberry Pi 4 is mostly a VideoCore GPU with an ARM attached. Emulating that GPU would be months of work and would still be wrong, because almost none of its behaviour is documented.

The observation that unlocked the project is that **RISC OS does not need a VideoCore. It needs the answers a VideoCore gives.** The ROM asks the firmware to power a device, and waits. It opens a sound service, and waits. It reads an EDID block to pick a screen mode. Every one of those is a question at a protocol boundary the ROM already speaks — and a question can be answered by a few hundred lines of software that model no silicon at all.

So the fork emulates only three kinds of thing: the CPU, the interrupt and timer plumbing the OS genuinely blocks on, and the handful of devices whose stock drivers write registers directly. Everything else is one of three substitutions:

1. **Impersonate the firmware** at the protocol boundary the ROM already uses.
2. **Supply signals from a host thread** instead of modelling timer blocks.
3. **Add a paravirtual doorbell** at an address the ROM never names, driven by a RISC OS module sitting on one of the OS's own extension points.

The records are unusually honest about the result. The approach made the boot possible; under TCG it did not make it dramatically faster. Chapter 8 is that account.



1. Don't build the machine you can borrow

Before the design rule, there was a decision about where to stand. The rule only makes sense on top of it.

Three ways to run RISC OS on a PC or a Mac

The project's design record lays out the options it weighed:

route	cost	what you get
write an emulator from scratch	"months"	total control, and every device to model yourself
a new HAL on QEMU's virt machine with VirtIO	a ROM of one's own	clean, and not the stock ROM anyone actually ships
QEMU's raspi4b machine, running the stock Pi ROM	days per blocker	a real CPU core for free, and a ROM people already have

The third won, for a reason stated plainly: *the CPU side of a Pi 4 sandbox costs nothing*. QEMU's TCG already translates ARM code, already models the Cortex-A72, and already runs it in AArch32. What it does not do is model a Raspberry Pi's peripherals the way RISC OS expects them.

The speed comparison the record keeps is worth having in view, because it sets the ceiling for everything later:

	MIPS
RPCEmu, interpreter	205
RPCEmu, dynarec	~920
QEMU TCG, register-heavy	~2,000
QEMU TCG, load/store-heavy	~500

TCG is roughly in the class of a good dynamic recompiler. That is fast enough to run a desktop, and — this becomes important in chapter 7 — slow enough that the cost of an emulated device access is small next to the cost of emulating the CPU around it.

The ROM is the fixed point

Everything that follows is constrained by one commitment made at the start:

no RISC OS-side source changes (we are not rebuilding the ROM)

That is a stronger constraint than it sounds. It means that when the ROM asks a question QEMU cannot answer, the fix has to be on the QEMU side — or in a module the ROM loads through its own sanctioned mechanisms. It rules out the easy answer of changing the driver that asks.

It also turned out to be the right constraint for a second reason, recorded later in the sound work: answering the ROM's questions *"is smaller than patching, and survives the next ROM release."* A patched ROM is a fork of RISC OS to maintain. An answered question is not.

The method: trace, don't theorise

The records are emphatic about how the blockers were found, and the emphasis comes from having been wrong.

The first diagnosis of why the display hung was a theory about VCHIQ — the VideoCore messaging interface that published work on other Pi models had named as the blocker. The theory was wrong. The design record's summary of the lesson:

Every hypothesis formed any other way ... was wrong.

The method that worked was to instrument QEMU's device layer, boot, and count what the guest actually touched. The record keeps the event volumes from successive fixes:

1,049,989 → 6,024 → 5,685,747 ...

Each number is the MMIO access count for a boot attempt. A number like 1 million concentrated on one register is not "a slow boot" — it is a spin loop, and the register it spins on names the device that is not answering. The first real blocker was found exactly that way: the HAL posted a request on mailbox channel 0 and then read `MAIL0_STATUS` **3.9 million times in twelve seconds**, with no timeout, waiting for a reply QEMU never sent.

That single observation is the template for the whole project:

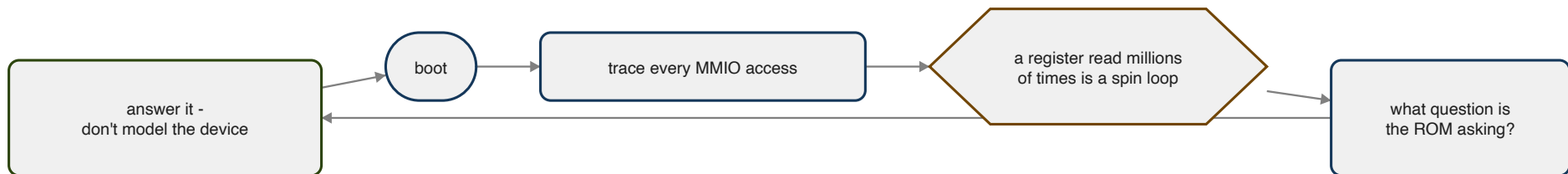
1. boot and trace
2. find the register the guest spins on
3. work out what question it is asking
4. answer the question — do not model the device

Reading before writing

A second habit mattered as much. Before the VCHIQ peer was written, its data structures were read **out of a live guest**: the shared-memory layout, the slot format, the handshake state, all observed rather than assumed from a header.

That turned out to matter again months later in the sound work, when the BCM2711's page-list encoding proved to differ from the one in Linux's header. The record's line on that is the best summary of why reading beats assuming:

Where the two differ, only one of them is the guest.



What "the machine you can borrow" does not include

Borrowing QEMU's `raspi4b` machine gets a CPU, a memory map and a set of peripheral models. It does not get peripheral models that behave the way RISC OS needs — they were written for Linux, which boots a Pi very differently. The companion [QEMU A72 walkthrough](#) is the account of each of those Linux-shaped assumptions. This document is about the decision of what to do once you have found one: model the device properly, or answer the question and move on.

The answer, almost every time, was the second. [Chapter 2](#) is where that became a rule.

2. As soft and fake as possible

The words, and when they were said

The rule is recorded in section 11 of the design record, under the heading "*The design principle that produced all of this*":

rather than emulating videocore; we should emulate the functions needed by risc os ... let's be as soft and fake as possible and emulate hardware only if we are desperate ... we probably do need interrupts though, the whole system is running off timers.

And the record's own gloss, directly beneath it:

Real interrupts where the OS genuinely depends on them. Functional shims everywhere else. The VCHIQ peer is about 250 lines and models no hardware; it answers questions.

The history is short and precise:

when	what
2026-09-09, 14:10	the channel-0 power fix lands — the first answered question
2026-09-09, 15:59	the VCHIQ peer lands; its commit message already says " <i>This is not a VideoCore</i> " and " <i>Refusing is deliberate</i> "
2026-09-09, 17:09	the design record is written down, with the rule in it

The principle was acted on before it was named. That is common with good design rules: the words arrive after the first instance proves them.

What it was a reaction to

Three things, all visible in the record.

The prospect of emulating VideoCore. Earlier published work on QEMU's older Pi models had named VCHIQ as the display blocker, and the first plan budgeted *"days"* for a VCHIQ peer with *"enough handshake that BCMVideo completes"*. The rule opens *"rather than emulating videocore"* because that was the road it was declining.

A diagnosis that had been wrong. The display hang had first been blamed on the wrong thing. Having been wrong once, the project stopped trusting theories about what the hardware must be doing.

Heavier alternatives on the table. A from-scratch emulator (*"months"*); prior art that modelled more hardware, such as a full USB network chip; and ROM maintainers' stated preference for a new HAL on QEMU's `virt` machine, which would need a ROM nobody has.

How the rule spread

It did not stay a one-off. It was restated, in new words, each time it met a new subsystem:

when	restatement	where
sound	<i>"a conversation, not a chip"</i>	SOUND.md
graphics	<i>"there is no GPU in QEMU — we are the GPU"</i>	GPUDESIGN.md
files	<i>"Do as much work in the host as possible, and as little in the guest as possible"</i>	FSDESIGN-V1.md
EDID	the timings table exists <i>"to be believed"</i>	bcm2835_property.c

The file-system restatement is a second-generation version of the rule, and worth pausing on. It is not about hardware at all. It is about *where logic lives*, and its argument is an asymmetry of tooling: code in the host is written and debugged with a modern compiler, a debugger and fast builds; code in the guest is ARM assembler or C under a 1990s toolchain in an emulator. The record puts it exactly:

Every line of logic moved from the second to the first is bought at a large discount.

The constraint that decided where fakes could go

The no-rebuild commitment from chapter 1 has a sharp consequence. If the ROM cannot change, a substitute can only sit in one of two places:

1. **at a firmware protocol the stock ROM already speaks** — mailbox, VCHIQ, property tags — so the ROM's existing drivers get answers they recognise; or
2. **on a sanctioned OS extension point** — a vector, a filing-system interface — through a module the ROM loads the ordinary way.

And a third principle keeps the second kind safe:

correctness never depends on coverage

Anything a substitute module does not handle passes through to the ROM's own code. So a substitute can be partial — it can take only the cases it is sure of — without ever making the system wrong. That principle is why the blitter in [chapter 5](#) can decline most of what it sees.

The first five fakes

These were the blockers on the road to a first desktop. Each is small, and each is a different flavour of "answer the question".

1. Mailbox channel 0: the reply is the request

The HAL asks the firmware to power on the USB host controller, then spins on the status register with no timeout — 3.9 million reads in twelve seconds.

The substitute is a 146-line peer whose entire logic is in one comment:

Since every device we model is always powered, the reply is the request.

In an emulator, every device is on. So "please power on device X" is answered by echoing it back as done. The plan had estimated about thirty lines. It can be proven without RISC OS at all: a test tool prints `00000080` where stock QEMU prints `TIMEOUT`.

2. VCHIQ: shake hands, then refuse everything

This is the purest example of the rule, and the most counter-intuitive.

RISC OS's sound driver calls `VCHIQ_Connect` unconditionally during start-up and blocks — *"no timeout, no deadline, and no register the guest re-reads."* The only way out is a genuine CONNECT message appearing in shared memory, followed by a doorbell interrupt.

The peer supplies exactly that: it marks the VideoCore side initialised, queues a CONNECT, and rings the VideoCore-to-ARM doorbell. And then **it answers every subsequent request to open a service by closing it.**

Why refusal is correct rather than lazy: accepting the audio or TV service would lead straight into further untimed waits. Declining leaves the OS's *"GPU mode available"* flag at zero, so the display driver falls back to the property channel — which QEMU already models completely. The README's note is that refusal *"is deliberate rather than lazy."*

The peer was later taught to accept selected services, once there was something real behind them — the record's phrase is *"the refusal was right until something stood behind it."* [Chapter 4](#) is what stood behind it.

3. Firmware answers on the property channel

The screen mode RISC OS picks depends on the monitor's EDID block, which on a Pi the firmware supplies. The substitute is a synthetic EDID table, answered on the property channel. Before it, the desktop came up at 640×256; after it, at a proper table of modes from 640×480 upward.

Tellingly, the *emulated* alternative was tried first and removed: an EDID EEPROM on the I2C bus. It failed because RISC OS probes that I2C address looking for a real-time clock, and reads the EDID bytes as clock data. On an emulated Pi with no clock, *"NACK is the correct answer."*

4. Read-as-zero stubs

On a Pi 4 only, RISC OS probes the PCIe root complex for the USB 3 chip. Against unmapped memory those probes took external aborts — fourteen of them in a 30-second boot. An "unimplemented device" region that simply reads as zero means *link down*, and the aborts dropped to one. The commit message has the best line in the set:

the difference between 'there is nothing here' and 'the bus is on fire'

The same stub sits over the Ethernet MAC. It does not save the boot on its own — the Ethernet driver dereferences a pointer it never filled in, *"which no amount of device modelling can prevent"* — so the fix there is in software too: a CMOS bit that unplugs the driver.

Modelling the USB 3 controller over PCIe was deliberately declined: *"a large project for a peripheral we want mainly as a keyboard."*

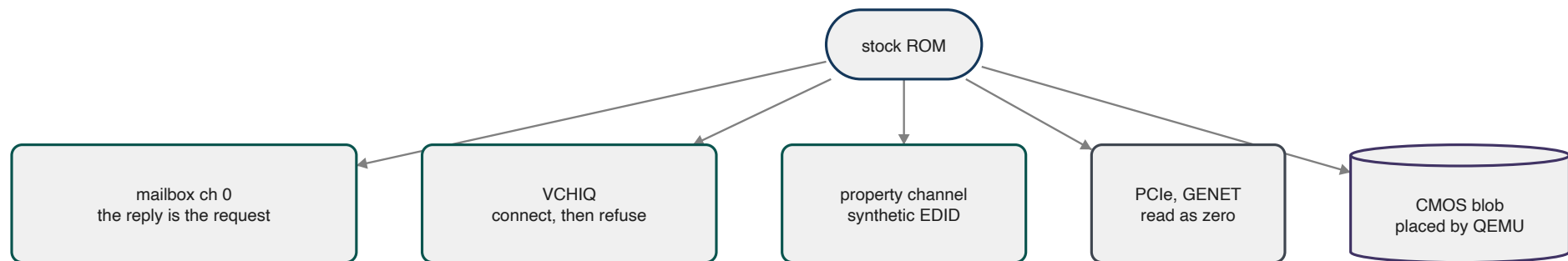
5. CMOS: QEMU is the firmware

A Pi has no CMOS chip. The firmware leaves a blob of settings in memory after the OS image, and the HAL reads it. Under emulation, the fork notes, *QEMU is the firmware* — so QEMU's generic loader simply places a CMOS blob at the address the ROM's header implies. It *"needs no device model at all."*

Two consequences follow, both good. The VideoCore firmware is never executed — the ROM is loaded directly — which also keeps Broadcom's firmware licence out of scope entirely. And later, CMOS persistence needed no NVRAM device either: the host file system saves it to the host after each write.

The five, side by side

blocker	what would normally be emulated	what was done	kind
channel 0	VideoCore power	echo the request	fake reply
VCHIQ	the VideoCore and its services	handshake, then refuse	protocol peer
EDID	a monitor on I2C	a synthetic table on the property channel	fake reply
PCIe, GENET	root complex, USB 3, Ethernet MAC	read as zero	stub
CMOS	NVRAM and firmware boot	a blob placed by QEMU's loader	host service



3. What the OS genuinely waits for

The rule has an exception written into it: *"we probably do need interrupts though, the whole system is running off timers."* This chapter is that exception — what was kept real, why, and the one place even the "real" parts turned out to be substitutes.

RISC OS runs on interrupts

RISC OS is interrupt-driven down to its bones. The cooperative desktop, the mouse, sound buffers, disc I/O and the centisecond ticker all depend on interrupts arriving, at the right rate, on the right lines. Faking an interrupt is not like faking a mailbox reply — the OS does not ask for it, it simply waits for it, and a late or missing one is a hang or a stutter rather than an error.

So interrupts are where the design says to stop faking. But it does not say to model the timer silicon.

Signals from one host thread

The design record states the approach:

RISC OS is interrupt-driven ... Rather than model those blocks, the fork provides the signals from one host thread.

A single high-resolution host thread — a Windows waitable timer, or a POSIX condition wait on macOS — fires deadlines. Each deadline raises the right interrupt line under QEMU's big lock. The timer *registers* the guest reads and writes stay, but they are cut down to what traces show is used:

- the ARM timer becomes *"a latch ... Nothing counts down here"*
- the SMI block becomes a latch where *"only CS does anything here"*
- vsync is *"what the VideoCore firmware does for the ARM on a real Pi, done by the high-resolution timer thread"*

The acknowledge registers *"stay as thin latches that only record what was written."* They exist so the ROM's drivers see the registers they expect; they do no timing of their own.

Why a thread, and not QEMU's own timers

Because QEMU's main-loop timers were not good enough. Measured:

	ticks per second	worst gap
QEMU main-loop timers	88.7	222 ms
dedicated host thread	100.0	11.6 ms
same thread, macOS POSIX branch	—	12.54 ms

A centisecond ticker that delivers 88.7 ticks a second is a clock that runs 11% slow, and a 222 ms gap is visible as a stutter. The thread hit the target rate exactly. The macOS branch was close enough that the heavier `mach_wait_until` primitive was measured as unnecessary and not used.

The cost: snapshots

Moving time into a host thread created one bug worth remembering. Restoring a saved machine brought back the timer compare registers but not the armed deadlines inside the host thread — so the ticker never fired again after a restore. The fix re-arms every deadline in the device's post-load hook.

It is the general shape of the risk: **state that lives outside the device model is state a snapshot does not know about.**

What stayed emulated, and why

Here is the part of the design that is easy to miss when describing it as "fake everything". A good deal was kept as real register-level emulation. The rule for what stayed is consistent:

Where a stock-ROM driver reads and writes real registers, the register model stays — cut down to what the traces show is used.

The devices that stayed

kept real	why
Cortex-A72 in AArch32 under TCG	it costs nothing; QEMU already had it
the GIC and the BCM2711 legacy interrupt and FIQ controller	the USB driver runs from the FIQ handler
system-timer compare semantics, including waiting for the wrap	the HAL's own re-arm loop depends on it
the DWC2 USB controller, HID keyboard and tablet	the stock ROM's only input path
the SD card controller	the reference boot medium
the I2C controller	the HAL probes for clocks, and needs a correct NACK
DMA, the framebuffer, the UART, the secondary-core stub	the display driver programs them directly

The FIQ path is the most striking entry. RISC OS's USB driver services the USB controller from the FIQ handler, not an IRQ, so a faithful fast-interrupt path is not optional — the keyboard and mouse do not work without it. That controller was built from the BCM2711 datasheet.

The network: cheaper emulation, not paravirtualisation

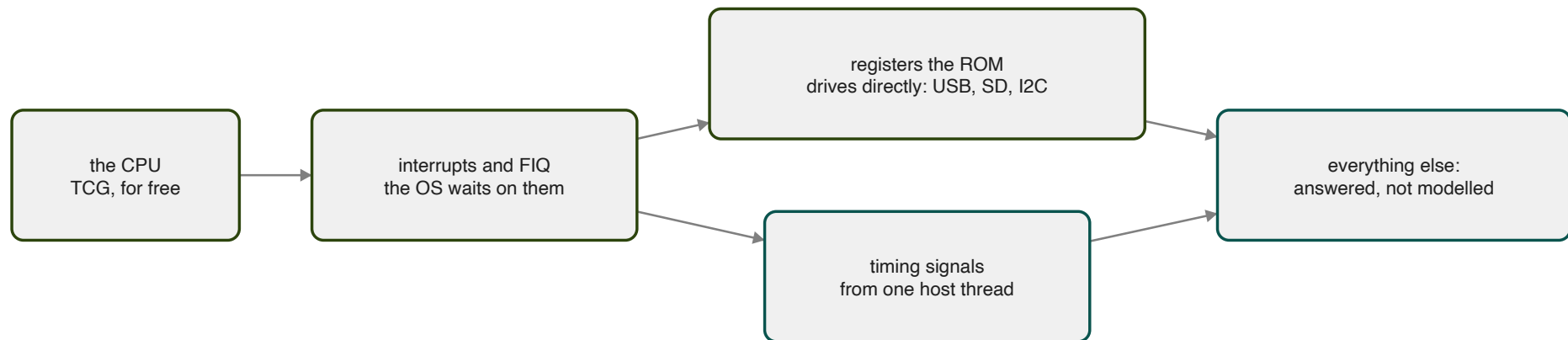
One entry is worth singling out because it shows the rule applied to a device that *did* need emulating.

The Pi 4's Ethernet MAC is GENET. Modelling it faithfully is a large job. But RISC OS's USB network driver has a generic CDC-Ethernet back end — and QEMU already has a USB network device. With a new `rndis=off` property it presents only the CDC-ECM configuration, and QEMU's user-mode networking answers DHCP.

The record calls this *"the soft answer and the one the design principle asks for."* It is, strictly, still emulation: the whole USB controller path is modelled. But it swaps a large, unmodelled device for a small, already-modelled one, which is the same instinct applied at a different level. The DHCP wait vanished and the boot reached the desktop in 27 seconds.

When "real" emulation still bites

Keeping emulation is not free either. Typing through the emulated USB keyboard drops or garbles keys under TCG unless the input is paced — the emulated controller cannot keep up with a host that types instantly. That shaped how the project's scripting and automation had to work, and a fast-typing crash in the USB driver remains open.



4. We are the GPU

Chapter 2 ended with the VCHIQ peer refusing every service. This chapter is what happened when the project decided some services were worth accepting — and how it accepted them without emulating a single piece of audio or display silicon.

The graphics record puts the stance in one line:

there is no GPU in QEMU — we are the GPU

Sound: a conversation, not a chip

The finding

The first thing the sound work established reframed the whole problem:

There is no audio hardware to emulate.

On a Pi 4, RISC OS does not drive a sound chip. Its sound driver opens a VCHIQ service called 'AUDS' and hands PCM buffers to the VideoCore firmware, which does the rest. The entire exchange is a handful of message types and one bulk data transfer. The record's summary is *"a conversation, not a chip."*

So the substitute is not a sound device. It is the other half of that conversation, living in the VCHIQ peer. The obvious hardware route — modelling the Pi's PWM or I2S audio — was explicitly declined, because it *"would be emulating hardware the guest never touches."*

The clock is the sound card

This is the cleverest idea in the sound work, and it only makes sense once you see what the conversation carries.

RISC OS generates sound at the rate the firmware *acknowledges* buffers. Each completion message tells the OS a buffer has been played, and the OS fills the next. So the rate of completion messages **is** the rate at which RISC OS makes sound.

The first attempts paced those acknowledgements from QEMU's virtual clock. The result was sound that played slowly:

spacing source	speed against real time
virtual-clock pacer, attempt 1	0.76×
virtual-clock pacer, attempt 2	0.71×
the host sound card	0.998×

The fix was to stop pacing at all. The peer now reports a buffer complete exactly when the host's audio back end has accepted it — so the host sound card, which genuinely runs in real time, becomes the clock the guest is driven by. The record:

the two quantities are the same quantity

It is a good example of the rule working at its best. A hardware model would have needed its own timing, and would have had to keep that timing in step with the host. Answering the conversation lets the host's real clock do the job directly.

Two bugs the conversation surfaced

Neither would have existed with a chip model, and both are instructive.

Slots were never recycled. The peer consumed the guest's message slots and never handed them back. Sound played perfectly and then stopped — after exactly 1,265 buffers, the moment the slots ran out. A bug with a precise, repeatable count is a resource leak, and this was one.

The page-list encoding was not Linux's. The BCM2711 encodes the physical pages of a bulk transfer in a 36-bit form that differs from the encoding in Linux's VCHIQ header. Following the header produced corrupt audio. Reading what the guest actually wrote produced sound. Hence the line quoted in chapter 1:

Where the two differ, only one of them is the guest.

The pointer: 'DISP', and the ROM does the pixel work

Why bother

Refusing the 'DISP' display service had a visible cost. With no hardware pointer, the kernel painted a software pointer instead — saving and restoring a 32×32 block of the screen underneath it on every single plot. That is work on every mouse movement, done in emulated ARM code.

The substitute

The peer answers the small subset of the dispmanx display protocol the pointer actually uses. The kernel then does what it does on real hardware: it renders the pointer shape itself and hands the "GPU" a finished image plus a position. The host window composites that image over the screen — on macOS in the Metal front end, on Windows in the DirectX 11 one.

The division of labour is the point:

The ROM does the pixel work.

The host never needs to understand RISC OS sprites, palettes or pointer shapes. It receives a finished ARGB image and a rectangle. The record calls that *"the pleasure of answering rather than emulating."*

Measured, and one honest cost

Enabling the service cut eleven refused open requests to a single accepted one, and a typical boot now sends 69 shape writes and around 356 moves. Each move costs three messages, which has not yet been measured against the software pointer it replaced.

The fidelity cost is small and specific: because the pointer is composited by the host window, it no longer appears in a QEMU screen dump. That matches real hardware, where a hardware pointer is not in the framebuffer either — but it surprised the tooling.

DMA: the one piece of GPU that stayed, made fast

Not everything graphical was replaced. The Pi's DMA controller is used by RISC OS's display driver for block copies, and the driver programs its registers directly — so by chapter 3's rule, it stayed emulated.

It stayed, and it was made fast. The model had refused transfers wider than 128 bits while the display driver believed they had completed, leaving a partly drawn desktop after every window move. Rows are now moved whole, and pattern fills take a fast path.

The graphics record treats that as *"the existence proof"*: if the host can execute the display driver's copies correctly and quickly, it can answer render requests. [Chapter 5](#) is where that idea went next.

```
manual layout line 3: unterminated quoted value
```

5. A blitter RISC OS never had

This chapter covers the blitter as an *instance of the principle*. Its full design — the module, the host code, damage tracking and the Windows and Mac displays — is the [Graphics and Sound walkthrough](#).

The premise

The sprints record opens the idea with a sentence that is half joke and half specification:

RISC OS draws with a blitter it does not have.

RISC OS's graphics driver interface offers two accelerated operations: copy a rectangle, and fill one. On the Pi, no code in the ROM ever asks the GPU to do either. The display driver declines fills outright, and sprites are plotted by the sprite module in ARM code, pixel by pixel. Under emulation, every one of those ARM instructions is translated and executed by TCG.

The Pi has no 2D blitter to emulate. So the fork **invented one**.

A device that models no hardware

The `riscos-blitter` device sits at a physical address the Pi's hardware abstraction layer never names. Its header is unusually direct about what it is:

It models no real BCM2711 hardware.

Its interface is a set of registers and a GO write, and each design choice has a stated reason:

Four design choices

choice	reason
registers plus a GO write, not a command block in memory	<i>"Eleven writes cost about a microsecond"</i> , whereas a block needs a physical address, which costs an OS call
the framebuffer addressed by byte offsets	no guest addresses to translate for the common case
the sprite source given as a guest <i>virtual</i> address	<i>"Letting the host walk the guest's page tables costs the guest nothing"</i>
the host does the work with <code>memset</code> and <code>memcpy</code>	synchronous, simple, and fast — see below

That third row is the idea that later reshaped HostFS, in [chapter 6](#).

A module that sits on the OS's own extension points

On the guest side, a small module called GVFill makes the device useful. It finds the device by mapping its physical page and checking a magic word, then claims two of RISC OS's own vectors: the graphics driver vector and the sprite vector.

It takes only cases it is certain of:

- **plain-colour rectangle fills**
- **sprite plots that are unmasked, the same pixel format as the screen, and unscaled**

Everything else passes straight through to the ROM's own code. That is the *"correctness never depends on coverage"* principle from chapter 2 in its purest form: the module can be as conservative as it likes, and the system stays correct. And it can be backed out instantly — `*RMKill GVFill` returns the machine to pure ROM drawing.

Why the host CPU, not the host GPU

A natural assumption is that host-side drawing should use the host GPU. It was measured, and it should not:

one real redraw on an M4	time
memset on the host CPU	~81 μ s
the same fill through Metal	~2.6 ms

The blit has to be synchronous — the guest expects the pixels to be there when the call returns — and submitting GPU work and waiting for it costs thirty times more than simply writing the bytes. For a fill, the fastest GPU is the CPU's `memset`.

Measured

	per sprite plot
ROM drawing in emulated ARM	490 μ s
through the blitter	80 μs
speed-up	6.1×
differing pixels, Mac	0 of 2,304,000

The first version managed 4.2×. Getting to 6.1× was a matter of removing fixed costs, the largest being an object-model lookup per sprite that fell from 43.9 μ s to 2.4 μ s.

Where the principle met its limit

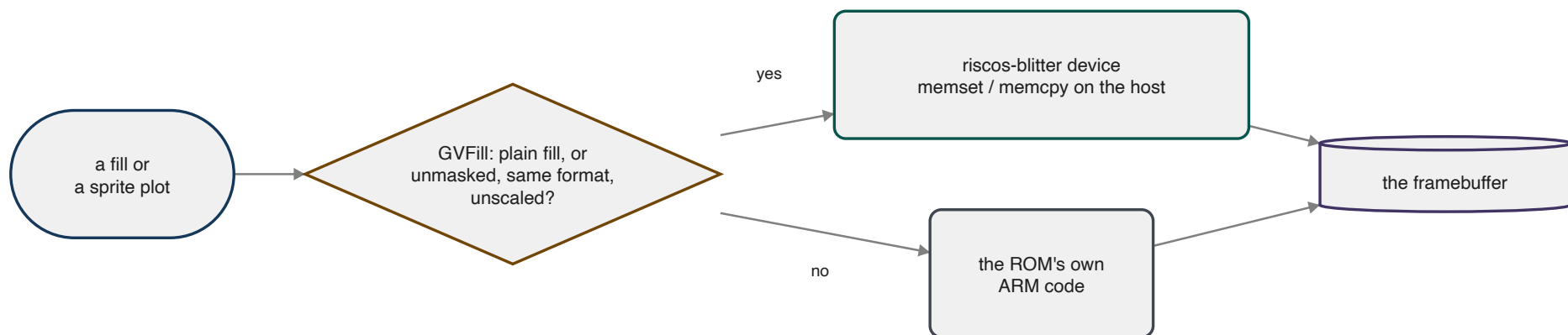
The blitter is also where the approach came closest to failing, and the records say so.

Masked sprites drew the desktop wrong. Accelerating sprites with masks was implemented and then backed out, after a bisect found the desktop differing by 484,513 pixels. With masked sprites taken back out, the blitter covers **94.3%** of sprite pixels. (The README still quotes an older 98.2%; that figure was measured on the build that was backed out.)

Coverage depends on configuration. At one Windows machine's default colour depth, the desktop scrolled with *zero* blitter operations — every plot fell through, because none matched the pixel format the module accepts.

Position on a vector matters. When GVFill was spliced into the ROM rather than loaded later, it *"never sees a sprite plot"* — a module further along claims the sprite vector first.

None of those is a correctness bug in the final design, because fall-through protects correctness. All three are coverage gaps, and the difference between the two is exactly what the principle was built to guarantee.



6. A doorbell and a filing system

This chapter covers the guest–host channel and HostFS as an *instance of the principle*. The full filing-system design is the [HostFS walkthrough](#).

Three dead ends first

The project needed a way to get files in and out of the guest. It tried the routes that model hardware or reuse existing protocols before building anything:

route	what happened
the emulated SD card	works, and remains the reference — but a disc image is a poor way to edit files from the host
LanManFS over QEMU's user networking	<i>"no guest code at all"</i> was the appeal; it died because the ROM speaks only SMB1 and the Windows host speaks only SMB2
an emulated USB stick backed by a host folder	the stick enumerated and was never mounted

The LanManFS failure is worth noting because it was the "softest" option — no new guest code whatsoever — and it failed on a protocol-version mismatch neither end could be persuaded to change.

The doorbell

What was built instead is the third substitution kind from the index: a **paravirtual device at an address the hardware abstraction layer never names**.

The filing-system record explains why that address is safe:

RISC OS on the Pi reads no device tree, so a hole the HAL does not name does not exist as far as it is concerned.

The device, `vmchannel`, has almost nothing in it:

- the guest writes a request block — a 64-byte header plus inline data — into its own memory

- it writes the block's address to a single command register
- the host does all the work **synchronously, inside that one MMIO write**, and the status register always reads "done"

The record lists what that design does *not* need, and it is a long list:

no ring, no interrupts, no reorder window, nothing to migrate beyond four registers

It is not free, and the record says that too: "A slow host disc stalls the vCPU." A synchronous doorbell means the guest CPU waits for the host's file system. Under emulation that is acceptable; it is a real cost to name.

Why a device rather than trapping the CPU

Earlier RISC OS emulators reached the host by intercepting SWIs — recognising a particular software-interrupt number and handling it outside the guest. The doorbell deliberately does not do that:

no CPU state is forged and any guest code could use it

Nothing in the guest's register state is faked. A module writes to a memory address, as a driver writes to any device. There is no hypercall, no secure-monitor call and no semihosting trap anywhere in the guest modules.

v0 → v1: moving the logic to the host

The first HostFS was shaped like POSIX. The guest module translated RISC OS filing-system calls into open/read/write/close requests, passed physical addresses to the host, and translated every 4 KiB chunk of guest memory itself. It worked, and it put RISC OS semantics — the hardest part — in guest code.

It also had a fallback that was dangerous by design. If translating an address failed, it used the identity mapping instead. The record is blunt:

Silent corruption is the designed behaviour of that fallback.

Version 1 inverted the design, applying the file-system restatement of the rule from chapter 2 — *do as much work in the host as possible*:

	v0	v1
what the module sends	POSIX-shaped requests	RISC OS's registers R0–R7, verbatim
addresses on the wire	physical, translated by the guest	logical, translated by the host
names, types, dates, errors	worked out in the guest	worked out on the host
stated goal	—	<i>"no RISC OS semantics left in the module"</i>

And the headline number: a 256 KiB read went from **130 doorbells to one**.

"The host walks the guest's MMU"

This is the idea v1 borrowed from the blitter. All guest addresses on the wire are *logical* — as the guest program sees them — and the host translates them by walking the guest's own page tables, the same way a debugger reads a running program's memory. The record names its precedents: QEMU's semihosting memory access, the 9p file system's metadata handling, and the property channel's DMA idiom, adopted *"because real silicon cannot walk guest page tables."*

That last clause is the whole principle again. A real device cannot do this. An emulator can, cheaply, so it should.

The bug that taught the most

Walking the guest's page tables exposed a genuinely confusing failure. Reads into ordinary application memory failed at address `0x9000`, even though the program could clearly use that memory.

The diagnosis took several theories to reach, and the eventual root cause is a good lesson in trusting instruments:

RISC OS maps application space lazily, and the debug walk was telling the truth the whole time.

The pages simply did not exist yet. RISC OS maps application memory on first touch, so a freshly allocated buffer the program has never written has no page table entries — and a debugger-style walk correctly reports that. A proof line touched every page from BASIC first, and the failure address moved

exactly as far as the touching had.

The fix: fault the buffer in, from the guest

The host cannot cause a guest page fault. So the division of labour is:

1. the host moves data page by page, and reports how far it got
2. on a short count, the **guest module touches one byte per page** — through the operating system's own abort handler, which maps the page
3. it retries once

Measured on a freshly allocated 256 KiB BASIC array: the first attempt moved 252 bytes; after touching, the full transfer went through. If an error survives the retry, it names the offset.

Honesty over silence

Two decisions in HostFS are the same decision, and they are worth stating together because they are the opposite of v0's fallback.

- An earlier version reported a failed transfer as success. The record: *"Silence is the worst possible failure here."*
- `*SetType` used to succeed without doing anything. The code comment: *"A silent lie is worse than an error."*

And the module refuses to load against an old host rather than falling back — because *"a silent fallback to the path that corrupts memory is worse than not loading."*



7. Booting from the host

The last substitution goes furthest: no disc image at all. RISC OS boots, reads its `!Boot` sequence and reaches the desktop from a directory on the host. This chapter covers it as the principle's end point; the full design is in the [HostFS walkthrough](#).

The chicken-and-egg problem

A filing system is a module. RISC OS looks for `!Boot` very early. So the filing system that holds `!Boot` must already be running when that search happens — and it cannot be loaded *from* a filing system that does not exist yet. The boot design record calls the soft-load route exactly what it is:

soft-loading it from a filing system is a trap

Splicing into a stock ROM, without rebuilding it

Chapter 1's first commitment was not to rebuild the ROM. The answer here keeps that commitment in spirit while relaxing its letter.

A tool, `mkrom.py`, **adds modules to the stock ROM's module chain without growing the image or touching its header**. New modules overwrite the chain's terminator and fit into slack space the image already contains — 268 KiB of it in the 5.31 ROM — while the footers stay byte-identical. The tool refuses if the slack runs out. The ROM is not recompiled; it simply gains a module at the end of the list it already walks at start-up.

The design record had originally proposed the opposite — growing the image and updating its size field — and assumed about 64 KiB of headroom. That version was never built; fitting into existing slack avoided moving the CMOS blob and touching the header at all. The result is a stock RISC OS image that happens to contain a HostFS module, so the filing system exists before `!Boot` is sought.

The decision about what goes into the ROM is deliberately minimal:

Only the filing system goes in the ROM

Everything else — including GVFill — loads from the host share afterwards. That keeps the spliced image small and the set of things that must be ROM-safe as short as possible, which matters, because ROM-safety turned out to be hard (see [chapter 8](#)).

CMOS on the share, too

Chapter 2's CMOS blob placed by QEMU's loader had one limitation: changes did not persist. Rather than add an NVRAM device, HostFS now claims the operating system's byte vector and **saves CMOS to the share after each write**. The persistence problem was solved by the filing system that was already there, without modelling any storage hardware.

The honest reading: what it actually bought

This is the part of the project's records most worth reading, because it contradicts an earlier prediction and says so.

The boot design had predicted that replacing emulated SD access with the doorbell would make booting *"one to two orders of magnitude"* faster. The crossings did fall by orders of magnitude. The boot did not:

	boot to desktop	host crossings
from the emulated SD card	15.7 s	2,434,544 SDHCI accesses
from HostFS	14.4 s	2,657 doorbells

Nine hundred times fewer crossings, and an 8% faster boot.

The reason is the insight chapter 1 set up. Under TCG, an MMIO access to an emulated device is not an exit to a hypervisor — it is an ordinary function call inside the emulator. The file-system record:

under TCG an MMIO access is a function call, not an exit ... under TCG it is a footnote

The 2.4 million SD accesses were cheap, because each was just a call. The boot is dominated by emulating the CPU running RISC OS, and no amount of I/O efficiency changes that. The 2,657 doorbells cost 0.103 s of host time — 39 µs each.

The record draws the right conclusion and does not overreach: under TCG the doorbell is a footnote, but under hardware virtualisation, where every device access *is* an expensive exit to the hypervisor, the same design becomes the argument. The boot design record was updated to correct its wall-

clock prediction.

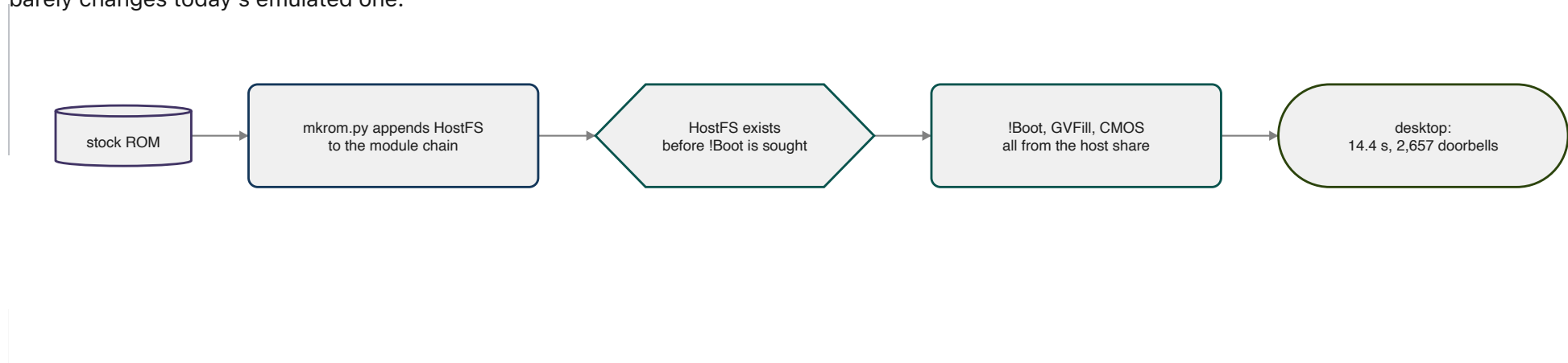
And the smaller costs are measured just as carefully. Scanning for typed file names on the host produced 722 misses, costing 33 ms of a 14.8-second boot — a number recorded precisely so nobody spends a week optimising it.

Why this matters for the principle

It would have been easy to present HostFS boot as a speed win and stop. The records instead establish the *true* case for it:

- it removes the disc image, so the host edits the guest's files directly
- it makes the boot path a host directory under version control
- it collapses millions of crossings into thousands, which will matter the moment the CPU is no longer emulated

That last point is the one worth carrying away. The fake-it-in-software approach optimises the right thing for a future accelerated host even where it barely changes today's emulated one.



8. The bill

The approach made RISC OS on an emulated Pi 4 possible, and in several places made it fast. It also cost things, and the project's records are candid about every one of them. This chapter collects them.

1. Silent failure is the characteristic failure mode

A hardware model that is wrong usually fails loudly — a bus error, a hang with a register named. A substitute that is missing or unconfigured often fails by doing nothing, and nothing looks the same as "not implemented".

Sound on Windows was silent for two days. Sound needs two command-line options to reach the host audio back end. Without them it *"fails quietly"*, and silence *"is indistinguishable from sound never having been built."* The lesson recorded:

a feature is not ported until the launcher that reaches it is ported too

The same shape appears elsewhere: v0 HostFS's identity-mapping fallback corrupted memory silently, and `*SetType` once succeeded without doing anything. The response in each case was to make the failure loud — refuse to load, return an error, name the offset.

2. The guest has to carry modules, and modules bring their own problems

Every paravirtual substitute needs a RISC OS module in the guest. That buys correctness through fall-through, and it creates a class of failure that pure emulation never has.

Four ways a module fails

problem	what happened
vector ordering	GVFill spliced into the ROM <i>"never sees a sprite plot"</i> , because a module later in start-up claims the sprite vector first
ROM-safety	a module in ROM may not write to its own image; claims must pass the private word; module sizes go in a different register; the C runtime relocates in place
shadowing	loading HostFS from a card's boot sequence replaced the ROM's version 2.00 with an older 1.01
version coupling	a v1 HostFS module refuses a v0 emulator — deliberately, but it couples releases

ROM-safety in particular became its own body of knowledge, recorded as a recipe because it was learned one failure at a time.

3. Acceleration puts correctness at risk

Fall-through protects correctness only as long as what *is* accelerated is exactly right, and that took several attempts:

- masked sprites *"drew the desktop wrong"* — 484,513 differing pixels — and were backed out
- an unexplained plot-action bit changed 3,468 pixels
- a Windows run found 2,688 differing bytes where the Mac found none
- an origin and clipping slip put every host-drawn sprite in the wrong place

Each was caught because the project compared host-drawn output against ROM-drawn output pixel for pixel. Without that comparison, several would have shipped.

4. Coverage depends on configuration

The blitter accepts only the pixel format it understands. At one Windows machine's default colour depth, the desktop scrolled with zero blitter operations. A cache of screen-mode constants read from the wrong place took none of 70 plots. The design is still correct in those cases — it just is not doing anything.

5. Fidelity

- **The pointer is gone from screen dumps**, because the host composites it — matching real hardware, but surprising the tooling.
- **Intermediate redraw states became visible**. Reading guest RAM on the host's clock shows states a real Pi displays for one scan-out. That reversed an earlier design note that *"a torn frame is exactly what a real monitor shows."*
- **Datestamps were an hour off under British Summer Time**. The fix added the host's UTC offset, on the reasoning that RISC OS keeps local time in a filestamp. A later design note argues the opposite — that RISC OS datestamps are UTC by definition, and the right fix is UTC on the wire with the time zone set in the guest. The code has not followed the note yet, so the question is open.

6. Portability to real hardware is nil for the paravirtual parts

The blitter *"models no real BCM2711 hardware"*. On a real Pi, GVFill and HostFS 2.x have nothing to talk to; they report that there is no host blitter or no doorbell and stand aside. That is by design, and it is still a cost: the substitutes are an emulator-only layer, and nothing about them helps RISC OS on silicon.

7. Assumptions that hold only under TCG

Writing guest RAM from the host, behind the CPU's back, is safe today because TCG has no cache model. The record flags the assumption explicitly:

If an HVF or KVM path ever appears, this needs revisiting.

That is the other side of chapter 7's argument. Hardware virtualisation would make the doorbell far more valuable — and would invalidate some of the shortcuts that made the doorbell simple.

8. Snapshots and state outside the device model

- host-thread timer deadlines were not restored until a post-load hook re-armed them
- the doorbell device keeps no migration state, while the blitter migrates its registers
- *"A snapshot decides its own ROM"* — a machine saved without HostFS comes back without it

9. Namespaces

The filing-system number HostFS uses is not allocated by RISC OS's maintainers, and host file names that Windows cannot store needed a private-use character mapping.

10. The records contradict themselves in places

A walkthrough built on design notes has to check them against each other, and these disagree in several places:

claim	the problem
blitter covers 98.2% of sprite pixels	measured on the masked-sprite build that was backed out; the current figure is 94.3%
the relocatable module area is identity-mapped	one file says it is, another says it is not; HostFS translates addresses rather than assuming

The contradictions, continued

claim	the problem
which peripheral window can hold a doorbell	one comment calls a region unreachable, another calls it <i>"provably live"</i> and uses it
the VCHIQ page-list format	a comment still describes the 12-bit form; the code uses the 36-bit one
HostFS module size target of 250–300 lines	the module is several times that
HostFS is whole-file only	an older README; streams and buffered I/O have since landed

None is a defect in the system. All of them are the ordinary drift of notes written at speed, and they are why every figure in this document was checked against its commit.

The method that kept it honest

Every item above was found, measured and recorded rather than discovered by a user. The records name the discipline that did it, and it is the same one that found the first blocker in chapter 1:

Every wrong turn this project has taken came from writing code before taking the measurement.

Tracing beat theorising every time.

And one refinement specific to measuring speed: alternate the variants being compared — A, B, A, B — rather than running one then the other, so a warming host cannot flatter whichever ran second.

Things that will bite

if you...	...this happens
add a substitute without making its absence loud	it fails silently, and looks unimplemented
port a feature but not the launcher flags that reach it	the feature does nothing on that host
accelerate a drawing case without a pixel comparison	a desktop that is subtly wrong

Things that will bite, continued:

if you...	...this happens
splice a vector-claiming module into the ROM	a later module may claim the vector first
write a ROM module like a soft-loaded one	ROM-safety failures, often late and obscure
assume guest memory is mapped	lazily mapped application space reads as absent
enable hardware virtualisation later	the host-writes-guest-RAM shortcuts need revisiting
trust a figure in the README	check it against the commit that measured it